

S e s s i o n 5

Selectors ขั้นสูง และประสิทธิภาพ

createSelector · createEntityAdapter · การวัดและปรับปรุงประสิทธิภาพ

หลักสูตร React-Redux · โปรเจกต์ AcadeMate

Dr.Watcharin Sarachai

ทำไมต้อง Memoize?

เมื่อ useSelector re-render บ่อยเกินไป

createSelector

สถานะ derived แบบ memoized ด้วย Reselect

createEntityAdapter

คอลเลกชันแบบ normalized พร้อม selectors ในตัว

Lab 5 และการวัดประสิทธิภาพ

วัด แก๊จ และยืนยันประสิทธิภาพ

Session 5 — วาระและเป้าหมายการเรียนรู้

เมื่อสิ้นสุด Session นี้ นักศึกษาสามารถ:

ส่วนที่ 1 — ทำไมต้อง Memoize?

เข้าใจว่า useSelector ก่อให้เกิด re-render ที่ไม่จำเป็นเมื่อใดและสำคัญอย่างไร

ส่วนที่ 2 — createSelector

เขียน selectors แบบ memoized ด้วย Reselect เพื่อคำนวณ derived data อย่างมีประสิทธิภาพ

ส่วนที่ 3 — createEntityAdapter

ทำ Normalize คอลเลกชัน ใช้ CRUD และ entity selectors ในตัว

Lab 5 — ปรับปรุง AcadeMate

ย้าย selectors ไปใช้ createSelector และทำ normalize studentsSlice ด้วย entityAdapter

ส่วนที่ 4 — การวัดประสิทธิภาพ

ใช้ React DevTools Profiler และ Redux DevTools ค้นหาและแก้ไขปัญหา re-render

ทำไมต้อง Memoize?

ทุกการเรียก useSelector รัน selector บน dispatch ทุกครั้ง สำหรับ derived data ที่ซับซ้อน ทำให้สิ้นเปลือง CPU และเกิด re-render ที่ไม่จำเป็น

เมื่อ useSelector Re-render บ่อยเกินไป

useSelector รันซ้ำทุกครั้งที่มี state เปลี่ยน หาก selector สร้าง object หรือ array ใหม่ทุกครั้ง React จะ re-render แม้ข้อมูลไม่เปลี่ยน



สร้าง Array ใหม่เสมอ

```
// Re-renders EVERY component that uses
// this selector on EVERY dispatch
export const selectHighAchievers =
  state => state.students.list
    .filter(s => s.gpa >= 3.5);
// ↑ .filter() always returns a NEW array
//   even if the contents are identical.
//   React sees new reference → re-render.
```



Memoized ด้วย createSelector

```
import { createSelector } from '@reduxjs/toolkit';

export const selectHighAchievers =
  createSelector(
    state => state.students.list, // input
    list => list.filter(           // result
      s => s.gpa >= 3.5
    )
  );
// Re-runs ONLY when list changes.
```

หลักการทำงานของ Memoization

createSelector เก็บ cache ผลลัพธ์ล่าสุด เปรียบเทียบ inputs ด้วย reference (===) หาก inputs เป็น reference เดิม จะคืน cache ทันที — ไม่คำนวณใหม่ ไม่สร้าง array ใหม่ ไม่ re-render

Reference Equality — เหตุใดจึงสำคัญ

React ใช้ `Object.is()` (เทียบเท่า `===`) เปรียบเทียบผลลัพธ์ selector สอง arrays ที่มีเนื้อหาเหมือนกันไม่ถือว่าเท่ากัน เว้นแต่จะเป็น reference เดียวกัน

```
// JavaScript reference equality demo
const a = [1, 2, 3];
const b = [1, 2, 3];
console.log(a === b);    // false — different objects in memory

// After a Redux dispatch that does NOT change students:
const prev = state.students.list;    // [studentA, studentB]
const next = state.students.list;    // same reference — no mutation

// Non-memoized selector:
prev.filter(s => s.gpa >= 3.5)        // new array []
next.filter(s => s.gpa >= 3.5)        // new array []
// [] === [] → false → React re-renders!

// Memoized selector (createSelector):
// input (list) didn't change reference → returns cached []
// cached [] === cached [] → true → no re-render ✓
```

RTK มี Reselect มาให้แล้ว: `createSelector` ถูก re-export โดยตรงจาก '@reduxjs/toolkit' — ไม่ต้องติดตั้ง package เพิ่ม

createSelector

เขียน selectors แบบ memoized สำหรับคำนวณ derived state และประกอบกันได้

createSelector — โครงสร้าง

createSelector รับ input selectors อย่างน้อยหนึ่งตัวและ result function จะรัน result function ใหม่เฉพาะเมื่อ input เปลี่ยน

```
import { createSelector } from '@reduxjs/toolkit';

// — Pattern: createSelector(inputSelectors..., resultFn) —

export const selectHighAchievers = createSelector(
  // Input selectors – run on every dispatch to get raw state
  state => state.students.list,           // input 1

  // Result function – runs ONLY when inputs change
  (list) => list.filter(s => s.gpa >= 3.5)
);

// Multiple inputs:
export const selectCoursesByStudent = createSelector(
  state => state.students.list,           // input 1
  state => state.grades.list,             // input 2
  (_, studentId) => studentId,           // input 3 (second arg)

  // Result receives all inputs in order
  (students, grades, studentId) =>
    grades.filter(g => g.studentId === studentId)
);
```

การประกอบ Selectors — นำกลับมาใช้และต่อยอด

แนวปฏิบัติที่ดีคือสร้าง selectors ที่ซับซ้อน โดยประกอบจากตัวที่เรียบง่ายกว่า แต่ละ selector ควรทำหน้าที่เดียว

```
// src/features/students/selectors.js
import { createSelector } from '@reduxjs/toolkit';

// — Primitive selectors (no memoization needed — no derived data)
export const selectStudentList = state => state.students.list;
export const selectStudentsStatus = state => state.students.status;
export const selectStudentsError = state => state.students.error;

// — Derived selectors (memoized — compute from list)
export const selectStudentCount = createSelector(
  selectStudentList, list => list.length
);

export const selectAverageGpa = createSelector(
  selectStudentList,
  list => {
    if (!list.length) return 0;
    const sum = list.reduce((acc, s) => acc + s.gpa, 0);
    return (sum / list.length).toFixed(2);
  }
);

export const selectHighAchievers = createSelector(
  selectStudentList,
  list => list.filter(s => s.gpa >= 3.5)
);
```


Parametric Selectors — เลือกตาม ID

เมื่อต้องการเลือกรายการตาม ID ให้เราสร้าง selector factory — ฟังก์ชันที่คืน selector ใหม่

รูปแบบ Selector Factory

```
// Returns a NEW selector per id call.
// Each gets its own memoization cache.
export const makeSelectStudentById =
  (id) => createSelector(
    selectStudentList,
    list => list.find(s => s.id === id)
  );

// Usage in a component:
const selector =
  useMemo(() => makeSelectStudentById(id), [id]);
const student = useSelector(selector);
```

การใช้กับ useSelector

```
// StudentRow.jsx
import { useMemo } from 'react';
import { useSelector } from 'react-redux';
import { makeSelectStudentById } from
  '../features/students/selectors';

function StudentRow({ id }) {
  // Memoize selector creation so we
  // don't create a new fn each render
  const selectStudent = useMemo(
    () => makeSelectStudentById(id),
    [id]
  );
  const student = useSelector(selectStudent);
  return <tr><td>{student?.name}</td></tr>;
}
```

ตัวอย่างล่วงหน้า: createEntityAdapter (ส่วนถัดไป) มี selectById และ entity selectors อื่นๆ มาให้ฟรี — factory pattern ใช้เฉพาะเมื่อไม่ได้ใช้ entityAdapter

createEntityAdapter

ทำ Normalize คอลเลกชัน Redux เพื่อการค้นหา $O(1)$ และการทำ CRUD reducers อัตโนมัติ

utility function ของ RTK ที่สร้าง reducer + selector พร้อมใช้งาน เพื่อจัดการ Collection แบบ Normalized อัตโนมัติ



ก่อนใช้ (Array ธรรมดา)

ปัญหาที่พบบ่อย:

- ค้นหา / อัปเดตด้วย `.find()` $\rightarrow O(n)$
- เขียน reducer ซ้ำ ๆ ทุก slice
- selector ต้องเขียนเองทั้งหมด
- โครงสร้าง state ไม่สม่ำเสมอ

```
// state.users = [  
  {id:'1', name:'Alice'},  
  {id:'2', name:'Bob'}  
// ]  $\leftarrow O(n)$  ทุก operation
```



หลังใช้ createEntityAdapter

สิ่งที่ได้รับ:

- โครงสร้าง `{ids, entities}` อัตโนมัติ
- CRUD operations ในตัว (`addOne`, `setAll...`)
- `selectAll`, `selectById` ให้ทันที
- ค้นหา / อัปเดตแบบ $O(1)$ เสมอ

```
// state.users = {  
  ids: ['1', '2'],  
  entities: {  
    '1': {id:'1'}  
  }  
// }  $\leftarrow O(1)$  ทุก operation
```

สไลด์ถัดไป: Normalized State, Setup, CRUD Methods, Selectors

สถานะ Normalized — ปัญหาที่แก้ได้

การเก็บข้อมูลเป็น arrays บังคับให้ค้นหาแบบ $O(n)$ สถานะ normalized ใช้ dictionary (object ตาม ID) เพื่อเข้าถึงแบบ $O(1)$



โครงสร้าง Array (Session 1-4)

```
students: {
  list: [
    { id:'1', name:'Alice', gpa:3.8 },
    { id:'2', name:'Bob',   gpa:3.1 },
    { id:'3', name:'Carol', gpa:3.5 },
  ]
}

// Find student by id —  $O(n)$ 
const s = list.find(s => s.id === id);

// Update student —  $O(n)$  find + mutation
const idx = list.findIndex(s => s.id === id);
if (idx !== -1) list[idx] = updated;
```



โครงสร้าง Normalized (Session 5)

```
students: {
  ids: ['1', '2', '3'],
  entities: {
    '1': { id:'1', name:'Alice', gpa:3.8 },
    '2': { id:'2', name:'Bob',   gpa:3.1 },
    '3': { id:'3', name:'Carol', gpa:3.5 },
  }
}

// Find student by id —  $O(1)$ 
const s = entities[id];

// Update student —  $O(1)$ 
entities[id] = updated;
```

ทำไมต้องใช้ createEntityAdapter?

ดูโค้ดจริงที่ต้องเขียนเอง (ซ้าย) เทียบกับโค้ดที่ใช้ adapter (ขวา)

✗ ก่อน — เขียน Reducer เอง (แบบ Array)

```
// state = { students: [] }

// ✗ UPDATE - ต้องค้นหา index เอง
updateOne: (state, action) => {
  const idx = state.students.findIndex(s => s.id ===
    action.payload.id);
  if (idx !== -1) {
    state.students[idx] = action.payload;
  }
},

// ✗ DELETE - ต้อง filter ทั้ง array
removeOne: (state, action) => {
  state.students = state.students
    .filter(s => s.id !== action.payload);
},

// ✗ FETCH - ต้องเขียน logic เก็บ array เอง
fetchFulfilled: (state, action) => {
  state.students = action.payload; // replace all
},
```

✓ หลัง — ใช้ createEntityAdapter

```
// state = { ids:[], entities:{} }

// ✓ UPDATE - adapter จัดการให้เลย
updateOne: (state, action) => {
  // ไม่ต้อง findIndex!
  studentsAdapter.updateOne(state, {
    id: action.payload.id,
    changes: action.payload
  });
},

// ✓ DELETE - 1 บรรทัด
removeOne: (state, action) => {
  // ไม่ต้อง filter!
  studentsAdapter.removeOne(state, action.payload);
},

// ✓ FETCH - 1 บรรทัด
fetchFulfilled: (state, action) => {
  studentsAdapter.setAll(state, action.payload);
},
```

createEntityAdapter — ตั้งค่าและ Initial State

createEntityAdapter คั้นชุด reducers และ selectors ที่สร้างไว้ล่วงหน้าสำหรับจัดการคอลเลกชันแบบ normalized

```
// src/features/students/studentsSlice.js - Session 5
import { createSlice, createEntityAdapter } from '@reduxjs/toolkit';
import { fetchStudents, addStudentAsync, updateStudentAsync, deleteStudentAsync } from './studentsThunks';

// 1. Create the adapter - sortComparer keeps ids[] sorted
const studentsAdapter = createEntityAdapter({
  selectId: student => student.id,
  sortComparer: (a, b) => a.name.localeCompare(b.name),
});

// 2. getInitialState() creates { ids:[], entities:{} }
// and merges in any extra fields you provide:
const initialState = studentsAdapter.getInitialState({
  status: 'idle',
  error: null,
});

const studentsSlice = createSlice({
  name: 'students',
  initialState,
  reducers: {},
  extraReducers: builder => { /* ← see next slide */,
});

export default studentsSlice.reducer;
```

createEntityAdapter — Reducer Methods ในตัว

adapter มี CRUD operations ที่ใช้ได้โดยตรงใน extraReducers ไม่ต้องค้นหา index เอง

```
extraReducers: builder => {
  builder
    // Fetch all — replace entire collection
    .addCase(fetchStudents.pending, state => { state.status = 'loading'; })
    .addCase(fetchStudents.fulfilled, (state, { payload }) => {
      state.status = 'succeeded';
      studentsAdapter.setAll(state, payload); // replaces ids + entities
    })
    .addCase(fetchStudents.rejected, (state, { payload }) => {
      state.status = 'failed'; state.error = payload;
    })
    // Add one
    .addCase(addStudentAsync.fulfilled, (state, { payload }) => {
      studentsAdapter.addOne(state, payload);
    })
    // Update one
    .addCase(updateStudentAsync.fulfilled, (state, { payload }) => {
      studentsAdapter.upsertOne(state, payload); // insert OR update
    })
    // Delete one
    .addCase(deleteStudentAsync.fulfilled, (state, { payload }) => {
      studentsAdapter.removeOne(state, payload); // payload = id
    });
},
```

createEntityAdapter — Selectors ในตัว

getSelectors() สร้างชุด selectors ที่จำเป็นจากโครงสร้าง normalized เชื่อมกับตำแหน่ง state ของ slice

Export จาก studentsSlice.js

```
// At the BOTTOM of studentsSlice.js
// Bind selectors to state.students
export const {
  selectAll:      selectAllStudents,
  selectById:     selectStudentById,
  selectIds:      selectStudentIds,
  selectEntities: selectStudentEntities,
  selectTotal:    selectStudentCount,
} = studentsAdapter.getSelectors(
  state => state.students
);
```

การใช้งานใน components

```
// GpaSummary.jsx
import {
  selectAllStudents,
  selectStudentCount,
} from '../features/students/studentsSlice';

function GpaSummary() {
  // selectAll returns an array
  // (sorted by sortComparer)
  const students = useSelector(selectAllStudents);
  const count    = useSelector(selectStudentCount);
  // ...
}
```

selectors เหล่านี้มี memoize มาแล้ว — selectAll ใช้ createSelector ภายใน สามารถประกอบกับ createSelector ของตัวเองเพื่อ derivation เพิ่มเติม

Lab 5 — ปรับปรุง AcadeMate

ย้าย `studentsSlice` ไปใช้ `createEntityAdapter` และแทนที่ inline selectors ด้วย `createSelector`

Lab 5 — ภาพรวมและไฟล์ที่เปลี่ยนแปลง

การเปลี่ยนแปลงล่าสุด — ทั้งหมดอยู่ใน data layer ไม่ต้องแก้ JSX

อัปเดต

`src/features/students/studentsSlice.js`

เพิ่ม `createEntityAdapter` — แทน `list[]` ด้วย `ids+entities` คง `extraReducers` ไว้

อัปเดต

`src/features/students/selectors.js`

แทน inline filter selectors ด้วย `createSelector` แบบ memoized

อัปเดต

`src/components/GpaSummary.jsx`

อัปเดต import path — `selectAllStudents` มาจาก `studentsSlice` แทน `selectors`

อัปเดต

`src/components/StudentTable.jsx`

ใช้ `selectStudentById` สำหรับค้นหา $O(1)$ ลบการเรียก `.find()` แบบ manual

Lab 5 — ขั้นตอนที่ 1: ย้าย studentsSlice ไปใช้ Entity Adapter

ขั้นตอนที่ 1: แทน list[] ด้วยโครงสร้าง normalized ไฟล์ thunk และ dispatch ใน components ไม่เปลี่ยน

```
// src/features/students/studentsSlice.js - Session 5
import { createSlice, createEntityAdapter } from '@reduxjs/toolkit';
import { fetchStudents, addStudentAsync, updateStudentAsync, deleteStudentAsync } from './studentsThunks';
const studentsAdapter = createEntityAdapter({
  sortComparer: (a, b) => a.name.localeCompare(b.name),
});
const initialState = studentsAdapter.getInitialState({ status: 'idle', error: null });
const studentsSlice = createSlice({
  name: 'students', initialState, reducers: {},
  extraReducers: builder => {
    builder
      .addCase(fetchStudents.pending, st => { st.status='loading'; })
      .addCase(fetchStudents.fulfilled, (st,{payload}) => {
        st.status='succeeded'; studentsAdapter.setAll(st, payload); })
      .addCase(fetchStudents.rejected, (st,{payload}) => { st.status='failed'; st.error=payload; })
      .addCase(addStudentAsync.fulfilled, (s,{payload}) => studentsAdapter.addOne(s, payload))
      .addCase(updateStudentAsync.fulfilled, (s,{payload}) => studentsAdapter.upsertOne(s, payload))
      .addCase(deleteStudentAsync.fulfilled, (s,{payload}) => studentsAdapter.removeOne(s, payload));
  },
});
export const { selectAll: selectAllStudents, selectById: selectStudentById,
  selectTotal: selectStudentCount } = studentsAdapter.getSelectors(s => s.students);
export default studentsSlice.reducer;
```

Lab 5 — ขั้นตอนที่ 2: Memoize Derived Selectors

ขั้นตอนที่ 2: แทน selectors ที่ไม่มี memoize ใน selectors.js ด้วย createSelector Primitive selectors คงเดิม

```
// src/features/students/selectors.js - Session 5
import { createSelector } from '@reduxjs/toolkit';
import { selectAllStudents } from './studentsSlice';
// — Primitive selectors (return scalars - no memoization needed)
export const selectStudentsStatus = state => state.students.status;
export const selectStudentsError = state => state.students.error;
// — Derived selectors (memoized - compute arrays or objects)
export const selectAverageGpa = createSelector(
  selectAllStudents,
  students => {
    if (!students.length) return '-';
    return (students.reduce((acc, s) => acc + s.gpa, 0) / students.length).toFixed(2);
  }
);
export const selectHighAchievers = createSelector(
  selectAllStudents,
  students => students.filter(s => s.gpa >= 3.5)
);
export const selectGpaDistribution = createSelector(
  selectAllStudents,
  students => ({
    high: students.filter(s => s.gpa >= 3.5).length,
    medium: students.filter(s => s.gpa >= 2.5 && s.gpa < 3.5).length,
    low: students.filter(s => s.gpa < 2.5).length,
  })
);
```

Lab 5 — ขั้นตอนที่ 3: อัปเดต Components

ขั้นตอนที่ 3: Components เปลี่ยนเฉพาะ import paths — hooks และ JSX คงเดิม

GpaSummary.jsx — import ที่อัปเดต

```
// Session 4 import:
// import { selectAllStudents } from '../features/students/selectors';

// Session 5 import (entity adapter exports it from the slice):
import { selectAllStudents, selectStudentCount }
  from '../features/students/studentsSlice';
import { selectAverageGpa, selectHighAchievers }
  from '../features/students/selectors';
```

StudentTable.jsx — ค้นหาแถว O(1) (การปรับปรุงเพิ่มเติม)

```
import { selectAllStudents, selectStudentById }
  from '../features/students/studentsSlice';

// Previously: students.find(s => s.id === id) → O(n)
// Now in a StudentRow child component:
function StudentRow({ id }) {
  const student = useSelector(state => selectStudentById(state, id));
  return <tr><td>{student.name}</td><td>{student.gpa}</td></tr>;
}
```

การวัดประสิทธิภาพและ DevTools

วัดจำนวน render จริง ค้นหา hotspots และยืนยันว่าการปรับปรุงได้ผล

React DevTools — Tab Profiler

React DevTools Profiler บันทึก components ที่ render และสาเหตุ ติดตั้งเป็น browser extension แล้วใช้แท็บ Profiler

1 เปิด DevTools → แท็บ Profiler

คลิก  ใน Chrome/Firefox DevTools → แท็บ Profiler → คลิกปุ่ม Record (วงกลม)

2 ดำเนินการใน AcadeMate

Dispatch action (เช่น เพิ่มนักศึกษา) เพื่อเรียก render ที่ต้องการวัด

3 หยุดบันทึก

คลิก Stop profiler แสดง flame graph ของทุก component ที่ render

4 อ่าน flame graph

สีเทา = ไม่ได้ render, สี = render แล้ว สูงกว่า = ใช้เวลามากกว่า คลิกแถบเพื่อดูสาเหตุ

5 ตรวจสอบ 'Render reason'

ในแถบข้าง: 'Props changed', 'State changed' หรือ 'Context changed' หากเห็น 'Hook changed' — useSelector ทำงาน

Redux DevTools — การวินิจฉัย Re-renders

Redux DevTools แสดงทุก action ที่ dispatch และ state diff ที่เกิดขึ้น ใช้รูปแบบเหล่านี้วินิจฉัยปัญหาประสิทธิภาพ



dispatch มากเกินไป

ทุกการกดแป้นในช่องค้นหา dispatch ให้ใช้ local state สำหรับ inputs — dispatch เฉพาะเมื่อส่ง



State diff ขนาดใหญ่

action fulfilled แทนที่ array ขนาด 10,000 รายการ ตรวจสอบว่าใช้ setAll ในขณะที่ upsertOne เหมาะกว่า



Action storm

actions หลายตัวติดต่อกันรวดเร็ว ใช้ batching (batch() ของ RTK) หรือรวม logic ไว้ใน thunk เดียว



Diff สะอาด

เฉพาะ entity ที่เปลี่ยนถูกอัปเดตใน entities id คงเป็น reference เดิม — selectors แบบ memoized จะไม่รันซ้ำ

รายการตรวจสอบการปรับปรุงประสิทธิภาพ

ปฏิบัติตามกฎเหล่านี้ตามลำดับ — Gain ส่วนใหญ่มาจากรายการบนสุด

สูง

เก็บ state input φόρμไว้ใน local (useState)

ห้ามเก็บทุกการกดแป้นใน Redux Dispatch เฉพาะเมื่อส่งเท่านั้น

สูง

Memoize selectors ที่ derive arrays หรือ objects

selector ที่ใช้ .filter, .map หรือ {} สร้าง reference ใหม่ — ห่อด้วย createSelector

สูง

ทำ Normalize คอลเลกชันด้วย createEntityAdapter

ค้นหา $O(1)$ เทียบกับ $O(n)$ สำคัญมากสำหรับรายการที่มี 100+ รายการ

กลาง

แยก components ขนาดใหญ่เป็น children ขนาดเล็กกว่า

ส่ง ID เป็น props ให้ children เลือกข้อมูลเอง ลดขอบเขตการ re-render

กลาง

ใช้ selectById ใน row/card components

แต่ละแถว re-render เฉพาะเมื่อ entity ของตนเปลี่ยน — ไม่ใช่เมื่อรายการที่ไม่เกี่ยวข้องเปลี่ยน

ต่ำ

พิจารณาใช้ React.memo สำหรับ leaf components

ห่อ pure presentational components ใช้เป็นทางเลือกสุดท้าย — แม้ selector memoization ก่อน

ตรวจสอบความรู้

คำถาม:

นักพัฒนามี createSelector selectors สองตัวที่ขึ้นอยู่กับ selectAllStudents เมื่อ GPA ของนักศึกษาหนึ่งคนเปลี่ยน:

```
const selectHighAchievers = createSelector(selectAllStudents, list => list.filter(s => s.gpa >= 3.5));  
const selectAverageGpa    = createSelector(selectAllStudents, list => list.reduce(...));
```

selectors ตัวใดรันซ้ำ และกี่ครั้ง? การเปลี่ยน student หนึ่งคนทำให้ทั้งสองคำนวณใหม่หรือไม่?

คำตอบ:

ทั้งสองรันซ้ำ แต่ตัวละครั้งเดียว

เมื่อ GPA ของนักศึกษาเปลี่ยน reference ของ list ทั้งหมดเปลี่ยน (Immer สร้าง array ใหม่) selectAllStudents จึงคืน reference ใหม่ ทำให้ selectors ทั้งสองหมดอายุ cache แต่ละตัวคำนวณครั้งเดียวและ cache ผลใหม่

ข้อสังเกตสำคัญ

หาก dispatch action ที่เปลี่ยนสิ่งอื่นใน store (ไม่ใช่ students) selectAllStudents คืน reference เดิม → selectors ทั้งสองคืนค่า cache ทันที → ไม่มีการคำนวณใหม่เลย

Session 5 — สรุป

แนวคิดหลักที่ครอบคลุมในSession 5:



ทำไมต้อง memoize

useSelector รันทุก dispatch หากไม่มี memoization derived arrays สร้าง reference ใหม่ → re-render ที่ไม่จำเป็น



การประกอบ selectors

สร้าง selectors ที่ซับซ้อนจาก primitives ที่เรียบง่าย แต่ละ selector ทำหน้าที่เดียวและทดสอบแยกได้



Entity selectors ในตัว

getSelectors() มี selectAll, selectById, selectTotal — มี memoize พร้อมใช้งานทั้งหมด



createSelector

รับ input selectors + result fn โดย result fn รันเฉพาะเมื่อ inputs เปลี่ยน — cache ผลลัพธ์ด้วย reference equality



createEntityAdapter

ทำ normalize คอลเลกชันเป็น {ids, entities} สำหรับ CRUD แบบ O(1) มี setAll, addOne, upsertOne, removeOne และอื่นๆ



ขั้นตอนการวัดประสิทธิภาพ

React DevTools Profiler → ค้นหา components ที่ re-render → Redux DevTools → ตรวจสอบ action diffs → แก้ไข

ตัวอย่างล่วงหน้า — Session 6: RTK Query

Session 6 แทนที่ manual thunks และ loading state ด้วย RTK Query — layer ดึงข้อมูลและ cache ที่ทรงพลังในตัว Redux Toolkit

`createApi`

กำหนด API service เดียวพร้อม base URL และ endpoints สำหรับ queries (GET) และ mutations (POST/PUT/DELETE)

`useQuery` hooks

hooks ที่สร้างอัตโนมัติ: `useGetStudentsQuery()` แทนที่ `useEffect` + `dispatch(fetchStudents())` + การตรวจสอบ status

`useMutation` hooks

`useAddStudentMutation()`, `useDeleteStudentMutation()` — สถานะ loading/error ในตัวสำหรับแต่ละ mutation

Cache และการ invalidation

RTK Query cache ผลลัพธ์ตาม endpoint + args Invalidate cache tags เพื่อเรียก refetch อัตโนมัติหลัง mutations

เอกสารอ้างอิงและอ่านเพิ่มเติม — Session 5

แหล่งข้อมูล	URL	ใช้ในสไลด์
RTK: createSelector	https://redux-toolkit.js.org/api/createSelector	สไลด์ 7-9, ขึ้นตอนLab 2
Reselect Documentation	https://reselect.js.org/	สไลด์ 4-5 (memoization)
RTK: createEntityAdapter	https://redux-toolkit.js.org/api/createEntityAdapter	สไลด์ 12-14, ขึ้นตอนLab 1
Redux Essentials Part 6	https://redux.js.org/tutorials/essentials/part-6-performance-normalization	Session 5 ทั้งหมด
React DevTools Profiler	https://react.dev/learn/react-developer-tools	สไลด์ 21 (การวัดประสิทธิภาพ)
Redux DevTools Extension	https://github.com/reduxjs/redux-devtools	สไลด์ 22 (state diffs)
Redux: Deriving Data with Selectors	https://redux.js.org/usage/deriving-data-selectors	สไลด์ 7-9
RTK: Performance Optimization	https://redux-toolkit.js.org/usage/usage-guide#using-createentityadapter-with-normalizr	สไลด์ 11-14